



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNM, INSTITUTO TECNOLÓGICO DE HERMOSILLO

Ingeniería en Sistemas Computacionales

Asignatura: Programación Web

Docente:

Moroyoqui Ibarra Jonathan Adrian

Integrantes:

Anaya Montoya Ruben Javier

Rivas Chay Hannah Valeria

Robles Carrillo Issac Heron

Grupo: S6A

Fecha de entrega: 29/05/2026

6.2 INTRODUCCION

¿Qué es el sistema?

MyM inStock es una aplicación web para administrar una tienda de abarrotes. Tiene una parte backend hecha en C# con ASP.NET Core que funciona como API, y una parte frontend hecha en React que es lo que ve y usa el usuario. Los datos se guardan en una base de datos SQL Server.

¿Qué problema resuelve?

Una tienda de abarrotes necesita saber cuánto tiene en existencia, qué vendió, qué compró y a quién. Sin un sistema, todo eso se maneja en papel o en Excel, lo que genera errores, pérdidas de información y mucho tiempo perdido.

MyM inStock resuelve eso permitiendo:

- Registrar ventas y que el inventario se actualice solo
- Registrar compras a proveedores y que el stock suba automáticamente
- Ver el historial de todo lo que entró y salió
- Controlar quién puede usar el sistema con usuarios y contraseñas

Objetivo del proyecto

Crear una herramienta sencilla y funcional que le permita a la tienda MyM llevar el control de su inventario, sus ventas y sus compras desde cualquier navegador, reemplazando los registros manuales y reduciendo los errores del día a día.

6.3 DOCUMENTACION TECNICA

Tecnologias utilizadas:

- Version de .NET y librerias principales (NuGet)

- .NET 8
- Microsoft.EntityFrameworkCore.Design 8.0.22
- Microsoft.EntityFrameworkCore.SqlServer 8.0.22
- Microsoft.EntityFrameworkCore.Tools
- Scalar.AspNetCore
- Swashbuckle.AspNetCore

- Version de React y librerias principales (npm)

- Vite + React 8.0.12
- Reduxjs toolkit 2.13.0
- Ag-grid 35.3.0
- Axios 1.16.1
- React router dom 7.15.1

- Version de SQL Server

- SQL Server 16.0.1180.1

Arquitectura:

- Descripción de la estructura del proyecto

- API backend hecha en C#, utilizada como controlador en el modelo MVC.
- Interfaz hecha en ReactJS, utilizada como View en modelo MVC.
- Base de datos en SQL Server, utilizada como modelo en el modelo MVC.

- Explicación de la comunicación entre front end y back end

El frontend (React) y el backend (ASP.NET Core) se comunican mediante peticiones HTTP. El frontend envía solicitudes al backend a través de una API RESTful, y el backend responde con los datos solicitados o confirma las acciones realizadas.

Flujo de comunicación

1. El frontend (React) hace una petición HTTP a una URL específica del backend (endpoint)
2. El backend (API) recibe la petición, la procesa y ejecuta la lógica necesaria
3. El backend devuelve una respuesta en formato JSON
4. El frontend recibe la respuesta y actualiza la interfaz de usuario

Base de datos:

- Descripción de cada entidad y sus atributos principales

1. Usuarios

Almacena los usuarios del sistema que pueden iniciar sesión y realizar operaciones como ventas, compras, etc.

Atributos principales:

- Id: Identificador único del usuario (PK)
- Nombre: Nombre(s) del usuario
- ApellidoPaterno: Apellido paterno del usuario
- ApellidoMaterno: Apellido materno del usuario
- Username: Nombre de usuario para iniciar sesión (único)
- Password: Contraseña del usuario
- Habilitado: Indica si el usuario está activo (true) o deshabilitado (false)

Relaciones: Un Usuario puede tener muchas Ventas, muchas Compras y muchos Tokens.

2. Token usuario

Almacena los tokens de autenticación generados para los usuarios al iniciar sesión.

Atributos principales:

- Id: Identificador único del token (PK)
- Token: Valor del token generado
- FechaCreado: Fecha y hora en que se creó el token
- FechaVencimiento: Fecha y hora en que expira el token
- UsuarioId: Identificador del usuario al que pertenece el token (FK)

Relaciones: Un Token pertenece a un Usuario.

3. PROVEEDORES

Almacena información de los proveedores que surten productos a la tienda.

Atributos principales:

- Id: Identificador único del proveedor (PK)
- Nombre: Nombre o razón social del proveedor
- Telefono: Número de teléfono de contacto
- Direccion: Dirección física del proveedor

Relaciones: Un Proveedor puede tener muchos Productos y muchas Compras.

4. Categorías

Clasifica los productos por tipo o familia.

Atributos principales:

- Id: Identificador único de la categoría (PK)
- Nombre: Nombre de la categoría (ej. Electrónica, Oficina)
- Descripcion: Descripción opcional de la categoría

Relaciones: Una Categoría puede tener muchos Productos.

5. Productos

Catálogo de productos que se venden y compran en la tienda, con control de inventario.

Atributos principales:

- Id: Identificador único del producto (PK)
- Nombre: Nombre del producto
- Descripcion: Descripción detallada del producto
- Marca: Marca o fabricante del producto
- Stock: Cantidad disponible en inventario
- PrecioVenta: Precio al que se vende al cliente
- PrecioCompra: Precio al que se compra al proveedor
- CategoriaId: Identificador de la categoría del producto (FK)
- ProveedorId: Identificador del proveedor del producto (FK)

Relaciones: Un Producto pertenece a una Categoría y a un Proveedor. Puede aparecer en muchos Detalles de Venta y muchos Detalles de Compra.

6. Ventas

Descripción: Registra la cabecera de cada venta realizada.

Atributos principales:

- Id: Identificador único de la venta (PK)
- Fecha: Fecha y hora en que se realizó la venta
- Subtotal: Suma de todos los subtotales de los detalles (sin impuestos)
- Iva: Impuesto calculado (generalmente 16% del subtotal)
- Total: Monto total a pagar (Subtotal + IVA)
- UsuarioId: Identificador del usuario que realizó la venta (FK)

Relaciones: Una Venta pertenece a un Usuario y tiene muchos Detalles de Venta.

7. Detallesventa

Detalle de cada producto vendido en una venta específica.

Atributos principales:

- Id: Identificador único del detalle (PK)
- VentaId: Identificador de la venta a la que pertenece (FK)
- ProductoId: Identificador del producto vendido (FK)
- Cantidad: Número de unidades vendidas
- PrecioVenta: Precio unitario del producto al momento de la venta (se congela)
- Subtotal: Cantidad $\times$ PrecioVenta

Relaciones: Un DetalleVenta pertenece a una Venta y a un Producto.

8. Compras

Registra la cabecera de cada compra realizada a proveedores.

Atributos principales:

- Id: Identificador único de la compra (PK)
- Fecha: Fecha y hora en que se realizó la compra
- Subtotal: Suma de todos los subtotales de los detalles (sin impuestos)
- Iva: Impuesto calculado (generalmente 16% del subtotal)
- Total: Monto total de la compra (Subtotal + IVA)
- UsuarioId: Identificador del usuario que registró la compra (FK)
- ProveedorId: Identificador del proveedor al que se compró (FK)

Relaciones: Una Compra pertenece a un Usuario y a un Proveedor, y tiene muchos Detalles de Compra.

9. Detallescompra

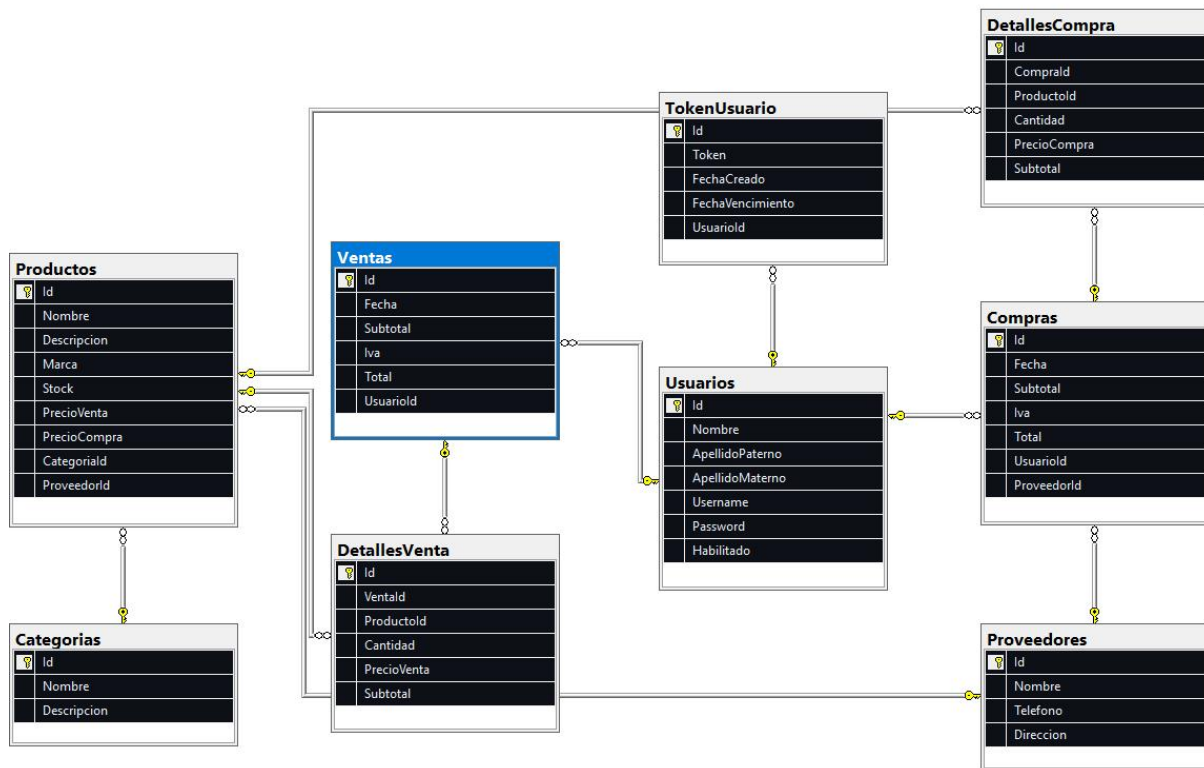
Detalle de cada producto comprado en una compra específica.

Atributos principales:

- Id: Identificador único del detalle (PK)
- CompraId: Identificador de la compra a la que pertenece (FK)
- ProductoId: Identificador del producto comprado (FK)
- Cantidad: Número de unidades compradas
- PrecioCompra: Precio unitario pagado al proveedor (se congela)
- Subtotal: Cantidad × PrecioCompra

Relaciones: Un DetalleCompra pertenece a una Compra y a un Producto.

- Relaciones entre tablas



API:

HTTP	Ruta	Descripción
AUTENTICACION		
POST	/api/auth/login	Iniciar sesión
USUARIOS		
GET	/api/usuarios	Obtener todos los usuarios
GET	/api/usuarios/{id}	Obtener usuario por ID
POST	/api/usuarios	Crear usuario
PUT	/api/usuarios/{id}	Actualizar usuario
DELETE	/api/usuarios/{id}	Eliminar usuario
CATEGORIAS		
GET	/api/categorias	Obtener todos los categorías
GET	/api/categorias/{id}	Obtener categorías por ID
POST	/api/categorias	Crear categorías
PUT	/api/categorias/{id}	Actualizar categorías
DELETE	/api/categorias/{id}	Eliminar categorías
PROVEEDORES		
GET	/api/proveedores	Obtener todos los proveedores
GET	/api/proveedores/{id}	Obtener proveedores por ID
POST	/api/proveedores	Crear proveedores
PUT	/api/proveedores/{id}	Actualizar proveedores
DELETE	/api/proveedores/{id}	Eliminar proveedores
PRODUCTOS		
GET	/api/productos	Obtener todos los productos
GET	/api/productos/{id}	Obtener productos por ID
POST	/api/productos	Crear productos
PUT	/api/productos/{id}	Actualizar productos
DELETE	/api/productos/{id}	Eliminar productos
VENTAS		
GET	/api/ventas	Obtener todas las ventas
GET	/api/ventas/{id}	Obtener venta por ID
POST	/api/ventas	Registrar nueva venta
COMPRAS		
GET	/api/compras	Obtener todas las compras
GET	/api/compras/{id}	Obtener compra por ID
POST	/api/compras	Registrar nueva compra

6.4 DISTRIBUCION DEL TRABAJO

Rivas Chay Hannah Valeria:

Básicamente lo que yo hice fue todo el desarrollo de backend, aka la API y los modelos de BD junto con la lógica de funcionamiento del frontend, aka que funciones los formularios, que las paginas estén conectadas entre si y que si se realice correctamente la comunicación entre backend y frontend.

Anaya Montoya Ruben Javier

Desarrollo FrontEnd:

Todo el diseño en css de login, home, users, categorias y proveedores.

Robles Carrillo Issac Heron

Me dedique al diseño una fracción del frontend en css tomando y modificando los módulos de Productos, ventas y compras.

6.5 RETROALIMENTACION DEL PROYECTO

Rivas Chay Hannah Valeria:

- Que fue lo mas dificil de desarrollar y como lo resolvieron

Lo mas complicado para mi fue realizar la lógica del frontend. No soy muy proficiente con React, no lo entiendo por completo y eso me dificulto conectar lo que pasaba en el front end con el back end, ya una vez que realice toda el área de usuarios fue mas sencillo. Lo resolví usando un montón de console logs para ver que era lo que se estaba enviando realmente, y poder dar con el lugar donde se rompía la comunicación de datos. Fue mas que nada prueba y error hasta que ya me familiarice con como funcionan las cosas.

- Que herramientas o recursos externos consultaron (documentacion, videos, etc.)

Stack overflow, esos foros de hace 8 años nunca fallan

- Que cambiarían o mejorarían del proyecto si tuvieran mas tiempo

Probablemente mejoraría la experiencia de usuario, por ahora por como ligue las paginas entre si la siento poco eficiente para acelerar el proceso que se realiza en la vida real

- Que aprendieron que no sabian antes de iniciar el proyecto

Aprendí a crear y poner en un uso una API, ademas de saber como funciona React.

Anaya Montoya Ruben Javier

- Que fue lo mas dificil de desarrollar y como lo resolvieron

En mi caso nada fue complicado, simplemente recordar como servia CSS

- Que herramientas o recursos externos consultaron (documentacion, videos, etc.)

Videos y un poco de ayuda de IA

- Que cambiarían o mejorarían del proyecto si tuvieran mas tiempo

Una mejor pantalla de Home

- Que aprendieron que no sabian antes de iniciar el proyecto

programacion en JS y creacion de APIs

Robles Carrillo Issac Heron

- Que fue lo mas dificil de desarrollar y como lo resolvieron

Realmente no tengo mucha practica con react ni con el frontend asi que todo lo que hice estuve consultando tanto con vídeos como con IAs para ayudarme a resolver cada problema con el que me encontraba, no hay ni un caso puntual que pueda señalar.

- Que herramientas o recursos externos consultaron (documentacion, videos, etc.)

Principalmente videos de youtube, foros e inteligencias artificiales como claude y gemini.

- ¿Qué cambiaría o mejoraría si tuviera más tiempo?

Pues en caso de haber tenido mas tiempo me hubiera gustado a reparar el problema que habia con la actualizacion de los datos y generar un diseño mas agradable a la vista

- ¿Qué aprendí que no sabía antes de iniciar?

Aprendí a utilizar mucho mejor Github y sus herramientas aledañas, tambien me familiarice mucho mas con visual studio y sus acoples con otras aplicaciones, aparte de mejorar mis conocimientos y habilidades de CSS.

6.6 RETROALIMENTACION DEL CURSO

Rivas Chay Hannah Valeria

- Que temas del curso les fueron mas utiles para el proyecto

Las clases donde fuimos armando poco a poco un API de ejemplo.

- Que temas sintieron que necesitaban mas practica o explicación

Senti que ocupaba mas explicación en como funciona React, o tal vez no mas explicación pero si ir mas lento y profundizar un poco mas

- Que le agregarían o quitarían al curso

Creo que agregaría un poco de contexto a javascript para luego empezar con React, y no aventarnos de lleno con React nomas

- Como calificarían el curso del 1 al 10 y por que

9, la verdad si me gusto mucho la clase y si senti que aprendí bastante, es nomas una que otra cosita que mencione en los anteriores puntos que me hacen no darle 10

- Comentarios adicionales para el profesor

La neta se la rifa profe, siga asi

Anaya Montoya Ruben Javier

- Que temas del curso les fueron mas utiles para el proyecto

El tema de JS y React

- Que temas sintieron que necesitaban mas practica o explicacion

En el de React

- Que le agregarian o quitarían al curso

Nada

- Como calificarian el curso del 1 al 10 y por que

10/10, el profe es honesto y justo con todos y explica muy bien los temas

- Comentarios adicionales para el profesor

Tremendo ganador

Robles Carrillo Issac Heron

- Que temas del curso les fueron mas utiles para el proyecto

Los temas de JavaScript, las APIs y React.

- Que temas sintieron que necesitaban mas practica o explicacion

Me hubiera gustado haber visto mas de nuevas tecnologías de la programación web orientandose mas a lo teórico que a lo practico.

- Que le agregarian o quitarían al curso

Agregaria mas actividades de tarea mas estructuradas para practicar, no estaria mal un classroom.

- Como calificarian el curso del 1 al 10 y por que

8 me parece un muy buen profesor y que tiene mucha experiencia en el campo y que sabe de lo que habla, solo siento que la estructura de las clases daba mucho campo a perderse o quedarse atrás y generaba que no pudieran continuar con las practicas de siguientes clases.

- Comentarios adicionales para el profesor

Hay gran campo de mejora como docente y tiene muchísimo potencial de ser de los mejores maestros de sistemas e informática.

También como dice mi compañero Ruben es muy honesto y accesible a la hora de hacer preguntas.